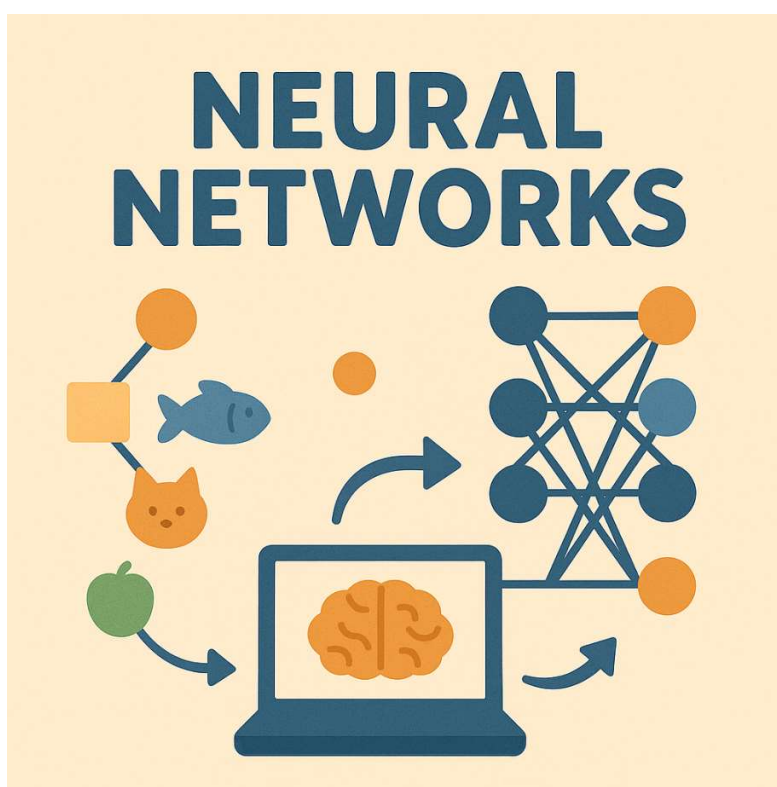


NF4 – XARXES NEURONALS



Autor: Generada ChatGPT

Curs d'Especialització en Intel·ligència Artificial i BigData

MP5072 Sistemes d'aprenentatge automàtic

Continguts

Introducció.....	5
Neurona biològica	6
Perceptró	7
Bias (biaix)	8
Procés d'entrenament d'un perceptró	8
Xarxa neuronal	10
Arquitectura bàsica	11
Funció d'activació.....	12
Funció pas (step).....	13
Funció sigmoide / logística	13
Funció Tanh (tangent hiperbòlica).....	14
Funció ReLU (R ectified L inear U nit)	15
Funció Leaky ReLU	15
Funció Softmax	16
Quadre resum de les funcions d'activació	18
Resum	19
Entrenament	21
Forward propagation.....	21
Funció de cost (loss function)	22
Backpropagation (retropropagació de l'error).....	23
Gradient descent (Descens de gradient)	23
Optimització i actualització de pesos	24
Regularització	26
Preprocessat.....	27

Disseny de l'arquitectura	28
Hiperparàmetres	29
Validació del model i seguiment de mètriques.....	31
Guardar i restaurar un model	32
Utilització de callbacks.....	32
Frameworks de Xarxes Neuronals.....	33
TensorFlow.....	33
PyTorch	33
Keras	34
Exeprimentació amb TensorFlow.....	34
Implementació d'una xarxa neuronal amb Keras	35
Exemple de Xarxa Neuronal - Regressió	35
Ajust d'hiperparàmetres.....	37
Número de capes ocultes (<i>Hidden layers</i>)	38
Número de neurones per capa.....	39
Learning Rate, Batch Size i altres hiperparàmetre	40
Tipus de Xarxes Neuronals.....	41
FNN /MLP (Feedforward Neural Network /Multilayer Perceptron).....	41
CNN (Convolutional Neural Network)	42
RRNN (Recurrent Neural Netowrks).....	43
LSTM (Long Short-Term Memory).....	44
GANs	44
Xarxes Autoencoder	45
Transformers.....	46
CNN Convolutional Neural Networks	47
Capa convolucional (Convolutional layer)	47

Hiperparèmtres importants: kernel, strid, padding	48
Apilar múltiples mapes de característiques	50
Capa de <i>Pooling</i> (<i>Pooling layer</i>)	50
Capa d'aplanament (<i>Flatten Layer</i>)	52
Resum	53
Transfer Learning	55
Annex.....	55
Referències	56

Introducció

La Natura ha estat, al llarg de la història, una font d'inspiració constant per a la humanitat. Moltes de les gran innovacions tecnològiques no han sorgit del no-res, sinó de l'observació atenta dels mecanismes i estratègies que la natura ha perfeccionat durant milions d'anys d'evolució. Aquest camp s'anomena **biomimètica**, i consisteix a estudiar sistemes naturals per replicar-ne els principis en solucions tecnològiques.

Per exemple l'aviació moderna es va inspirar en els ocells; els sistemes de sonar i radar tenen el seu origen en l'ecolització dels ratpenats; el famós velcro utilitzat en els vestits espacials i inspirat a la flor del card especialitzada en enganxar-se a la roba o al pèl dels animals.

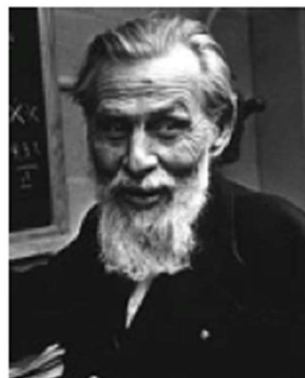
Aquesta mateixa lògica és la que va donar lloc al naixement de les xarxes neuronals artificials, un dels pilars fonamentals del Machine Learning modern (**Deep Learning**). Les xarxes neuronals són models computacionals inspirats en el funcionament del cervell humà format per xarxes de neurones biològiques. Alguns investigadors no els hi agrada parlar d'aquesta analogia biològica per la gran diferència que hi ha en el funcionament d'una neurona biològica i un **perceptró** (neurona artificial).

És cert que el funcionament dels dos elements és totalment diferent, però a mi m'agrada l'analogia amb el fet de que el treball d'una sola neurona no té cap "valor", però el treball conjunt de moltes neurones permet la realització de funcions extraordinàriament complexes i és aquí a on l'analogia té sentit. **Moltes unitats simples que , treballant conjuntament, són capaces d'aprendre patrons complexos a partir de dades.**

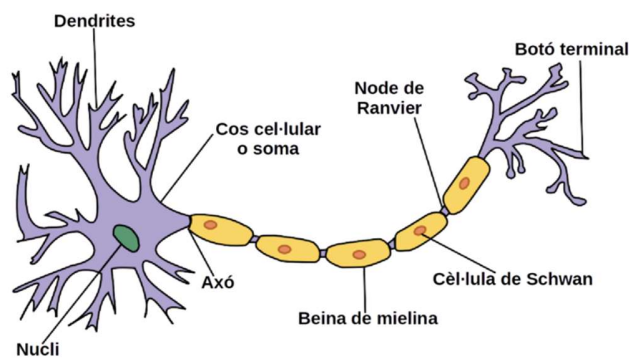
Les **xarxes neuronals** s'han convertit en una eina molt eficaç d'aprenentatge automàtic perquè són **capaces d'adaptar-se a molts problemes (regressió, classificació, llenguatge natural, imatges, so,...)**

El concepte de Xarxa Neuronal apareix per primera vegada en el 1943 de la mà del neuropsicòleg Warren McCulloch i el matemàtic Walter Pitts, en el seu treball de referència "[A Logical Calculus of Ideas Immanent in Nervous Activity](#)".

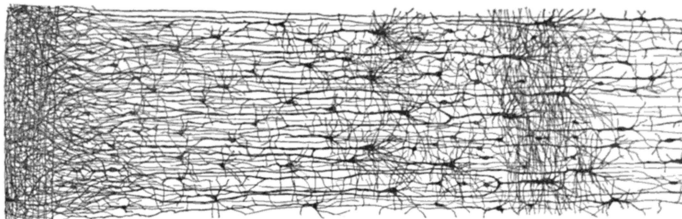
A la imatge tenim en Warren a l'esquerra i en Walter a la dreta.



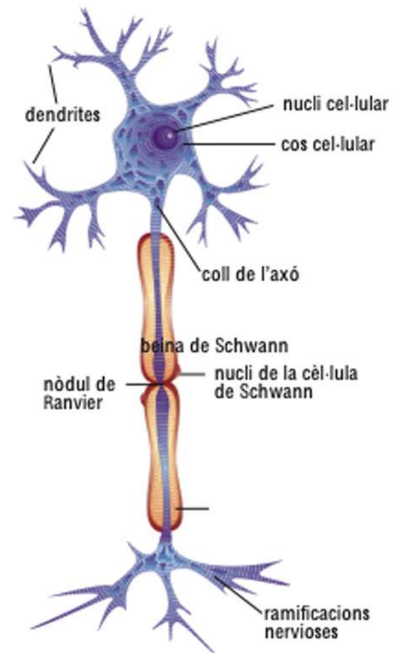
Neurona biològica



Wikipèdia



Art de la citoarquitectura del còrtex humà. Wikipèdia



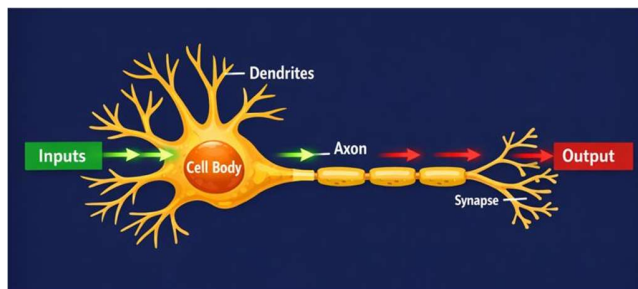
Enciclopèdia Catalana (Enciclopedia.cat)

Les parts bàsiques d'una neurona biològica són: **dendrites**, **axó** i les **ramificacions nervioses** o botons terminals.

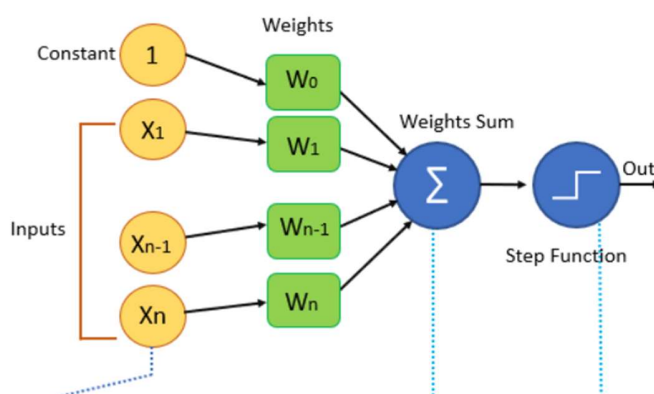
El funcionament simplificat d'una neurona és el següent, rep la informació a través de les dendrites i el cos neuronal (soma) processa la informació, integra els senyals rebuts i decideix si el senyal és transmès per continuar mitjançant el seu axó arribant fins a les ramificacions nervioses que passen aquesta informació a les dendrites d'una altra neurona mitjançant la **sinapsi**. En termes biològics no existeix un contacte directe entre neurones ja que la senyal es transmet mitjançant neurotransmissors químics.

<https://www.youtube.com/watch?v=SBia0Zv82VQ> (vídeo explicatiu de la Sinapsi neuronal).

Depenent de la naturalesa i la intensitat d'aquests senyals d'entrada, el cervell els processa i decideix si la neurona s'ha d'activar ("disparar") o no.



Perceptró



El diagrama següent mostra l'esquema d'un perceptró, és l'arquitectura més simple de xarxa neuronal creada per Frank Rosenblatt (psicòleg americà) el 1957.

Aquesta neurona artificial és lleugerament diferent per la proposada per McCulloch i Pitts. En aquesta nova neurona les **entrades** ($X_1, X_2, \dots, X_n$) i la **sortida** són números i no valors binaris (activat/esactivat) com proposaven McCulloch i Pitts.

Cada **entrada** està associada a un **pes** ($W_1, W_2, \dots, W_n$) i es calcula la funció lineal sumant la ponderació de les entrades amb els seus pesos ($W_1 \cdot X_1, W_2 \cdot X_2, \dots, W_n \cdot X_n$).

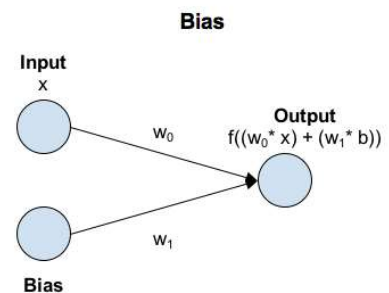
Finalment aquesta suma es passa a una **funció d'activació**/escalonada, que és la que produeix la sortida. Podríem dir que estem davant d'una funció logística vista en un model de regressió logística.

La **funció d'activació** és el **mecanisme de presa de decisions** de les xarxes neuronals. Aquesta funció produeix una sortida binària, per això a vegades s'anomena funció d'eglaó binària (step function). Si el valor és >0 , la classificació assignada és 1 o True. Si el valor és <0 , la classificació assignada és 0 o False.

Bias (biaix)

El biaix permet desplaçar la funció d'activació afegint una constant.

En les xarxes neuronals, el biaix es pot entendre com l'equivalent al terme constant d'una funció lineal, mitjançant el qual la recta es desplaça segons el valor de la constant.

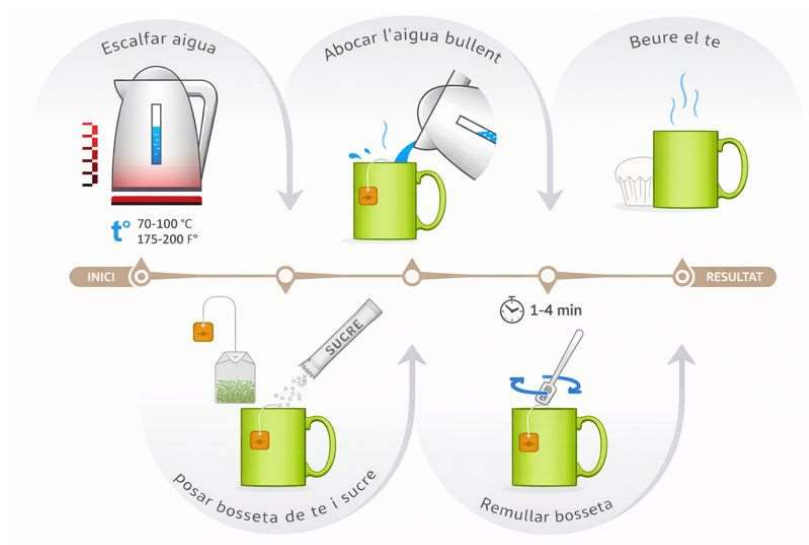


En un escenari amb biaix, l'entrada a la funció d'activació és 'x'

multiplicat pel pes de connexió ' w_0 ' més el biaix multiplicat pel seu pes de connexió ' w_1 '. Això té l'efecte de desplaçar la funció d'activació una quantitat constant : $b \cdot w_1$

Procés d'entrenament d'un perceptró

Considerem l'exemple de “Preparar un te” com a objectiu.



Aquest exemple es pot veure com un perceptró que prepara un bon te.

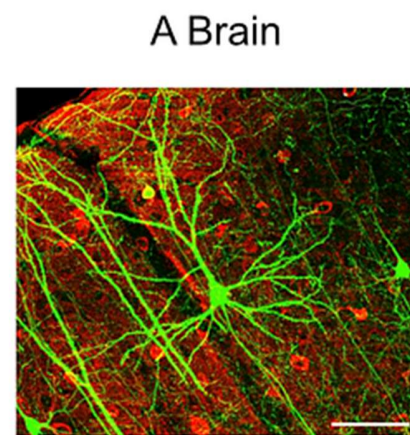
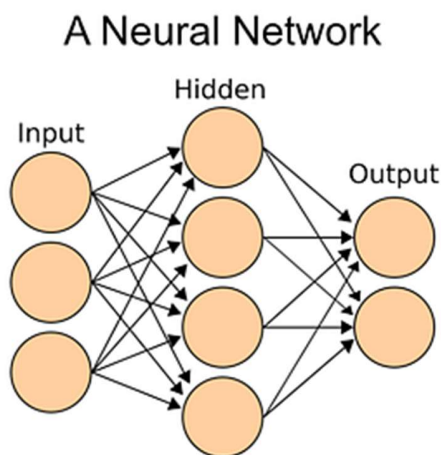
1. El primer pas és escalfar l'aigua fins arrencar el bull (100 °C) i abocar l'aigua a la tassa.
2. Afegir la bosseta de te i el sucre
3. Esperar 1-4 min. per obtenir el gust i l'aroma perfectes.
4. Finalment, remenar el te i retirar la bosseta.
5. Comprovació de la sortida

- Si la **sortida** és un **bon te**, no cal fer cap canvi. **FINAL**
- Si la **sortida** és dolenta, cal fer **retropropagació** (*backpropagation*) per canviar la temperatura inicial de l'aigua, la quantitat de sucre, la quantitat d'aigua o el temps d'espera remullant la bossa amb l'aigua. Després es torna a comprovar la sortida (pas 5)

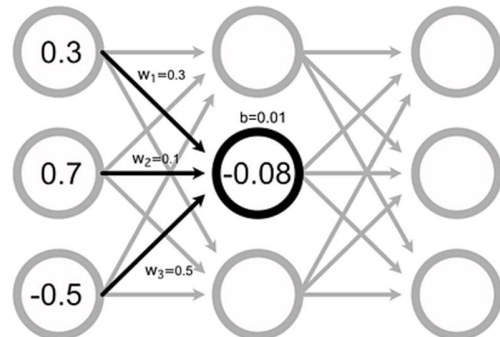
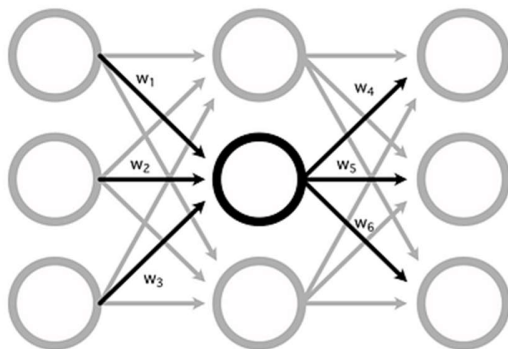
Aquí, l'**entrada** es tracta com l'**aigua** i els **pesos** es consideren com la **quantitat de sucre** i el **temps de la bosseta amb l'aigua**. Cal ajustar els pesos adequadament fins obtenir un **bon te**.

Xarxa neuronal

De la mateixa manera que una sola neurona poca feina pot resoldre. En el cas d'un perceptró passa el mateix. El que es sol fer és unir/connectar diferents perceptrons per resoldre diferents operacions més complexes.



[Xarxa neuronal i cervell humà](#)



Donades les següents entrades el càlcul de la segona neurona de la segons capa és:

$$resultat = (0.3 \cdot 0.3) + (0.7 \cdot 0.1) + (-0.5 \cdot 0.5) + 0.01 = -0.08$$

Arquitectura bàsica

Com es veu en la representació de les figures anteriors una xarxa neuronal bàsica és la connexió de diferents neurones connectades entre si i organitzades per capes seqüencials.

Capa d'entrada (Input layer)

La capa d'entrada rep les dades brutes del domini. No es realitza cap càlcul en aquesta capa. Els nodes aquí simplement transmeten la informació (característiques) a la capa oculta.

Capa oculta (Hidden layer)

Com el seu nom indica, els nodes d'aquesta capa no estan exposats. Proporcionen una abstracció a la xarxa neuronal. En una xarxa hi poden haver diferents capes ocultes i quan més capes/neurones tingui la xarxa, major serà la seva capacitat per trobar relacions complexes.

La capa oculta **realitza tot tipus de càlculs sobre les característiques introduïdes** a través de la capa d'entrada i transfereix el resultat a la capa de sortida.

Capa sortida (Output layer)

És la capa final de la xarxa és la que porta la informació apresada a través de la capa oculta i, com a resultat, ofereix el valor final.

La capa de sortida normalment utilitzarà una **funció d'activació diferent de la de les capes ocultes**. L'elecció d'aquesta funció depèn de l'objectiu o del tipus de predicció feta pel model.

El flux de la informació segueix una sola direcció, des de la capa d'entrada fins a la capa de sortida passant per les capes ocultes.

Cada neurona està connectada a totes les neurones de la capa següent, això és el que s'anomena xarxa densament connectada o **full connected**.

Exemple Graus (Cº) a Fahrenheit (Fº):

https://www.youtube.com/watch?v=iX_on3VxZzk&list=PLZ8REt5zt2Pn0vfJjTAPaDVSACDvnuGiG

Funció d'activació

Si ens mirem com es realitza el càlcul d'un perceptró veiem que matemàticament és idèntic a una **regressió lineal**: variables d'entrada multiplicades per uns pesos més un biaix.

$$Y = \beta_1 \cdot X_1 + \beta_0$$

Si la nostra xarxa neuronal fos una combinació de milers de milions d'aquests perceptrons el model resultant continuaria essent només una combinació lineal.

Per solucionar-ho el que fem és utilitzar el que s'anomena una funció d'activació.

La **funció d'activació** és el que **trenca la linealitat** del model canviant així la sortida i aquí rau l'èxit de les xarxes neuronals. D'aquesta manera podem partir l'espai de les dades de forma no lineal

La funció d'activació decideix si una neurona s'ha d'activar o no. Això vol dir que decidirà si l'entrada de la neurona a la xarxa és important o no en el procés de predicció mitjançant operacions matemàtiques simples.

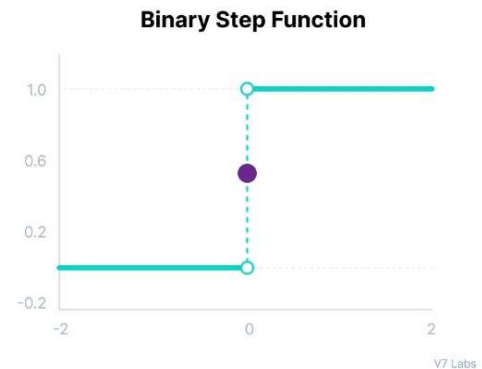
La funció és una mena de filtre que si la sortida no arriba a un cert llindar, no deixarà que surti

Totes les capes ocultes solen utilitzar la mateixa funció d'activació. Tanmateix, la capa de sortida normalment utilitzarà una funció d'activació diferent de la de les capes ocultes. L'elecció depèn de l'objectiu o del tipus de predicció feta pel model.

Podríem pensar que si utilitzem aquestes funcions d'activació estem perdent informació. Això seria així si utilitzéssim una sola neurona, però aquestes funcions d'activació tenen sentit quan combinem diverses neurones amb diverses capes

Funció pas (step)

Aquest tipus de funció és la més simple i actua com un interruptor depenent d'un cert llindar. L'entrada que s'introdueix a la funció es compara amb el llindar i si aquest és superior, la neurona s'activa, en cas contrari es desactiva. Això provoca que la sortida no passa a la següent capa.



Funció sigmoide / logística

Aquesta funció pren qualsevol valor real com a entrada i retorna valors en el rang de 0 a 1.

Com més gran sigui l'entrada (més positiva), més a prop estarà el valor de sortida d'1, mentre que com més petita sigui l'entrada (més negativa), més a prop estarà la sortida de 0.

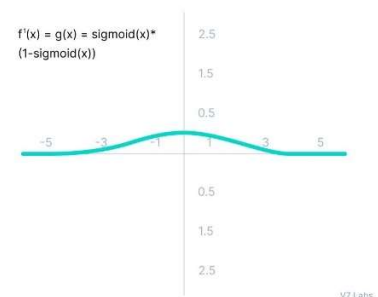
$$f(x) = \frac{1}{1 + e^{-x}}$$

La funció sigmoide és una de les funcions d'activació més utilitzades:

- S'utilitza habitualment per a models on hem de predir la probabilitat com a sortida. Com que la probabilitat de qualsevol cosa només existeix entre el rang de 0 i 1, sigmoide és l'elecció correcta a causa del seu rang.
- La funció és diferenciable i proporciona un gradient suau, és a dir, evita salts en els valors de sortida. Això es representa amb una forma de S de la funció d'activació sigmoide.



La derivada de la funció sigmoide és una funció a on veiem que el rang significatiu va de -3 a 3. Això significa per valors superiors a 3 o inferiors a -3, la funció tindrà gradients molt peits. Això implica que a mesura que el valor del gradient s'acosta a zero, la xarxa deixa d'aprendre.

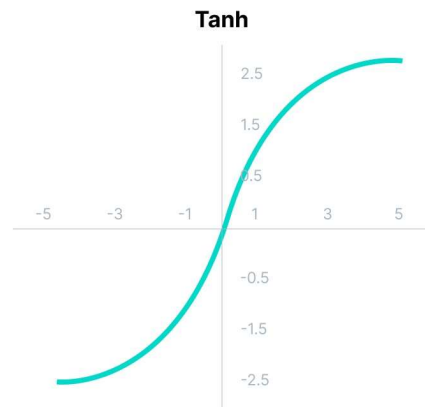


Funció Tanh (tangent hiperbòlica)

La funció Tanh és molt similar a la funció d'activació sigmoide/logística, i fins i tot té la mateixa forma de S, però el seu rang de sortida va de -1 a 1.

En aquesta funció quan més gran sigui l'entrada (més positiva), més a prop estarà el valor de sortida d'1, mentre que com més petita sigui l'entrada (més negativa), més a prop estarà la sortida de -1.

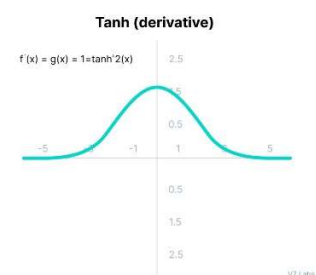
$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$



Els avantatges d'utilitzar aquesta funció d'activació són:

- La sortida de la funció d'activació de tanh està centrada en zero; per tant, podem fàcilment mapejar els valors de sortida com a fortament negatius, neutres o fortament positius.
- Normalment s'utilitza en capes ocultes d'una xarxa neuronal, ja que els seus valors es troben entre -1 i 1; per tant, la mitjana de la capa oculta resulta ser 0 o molt propera. **Ajuda a centrar les dades i facilita molt l'aprenentatge per a la següent capa.**

La derivada de la funció tanh pateix el mateix problema que en la sigmoide dels gradients nuls. A més, el gradient de la funció tanh és molt més pronunciat en comparació a la sigmoide



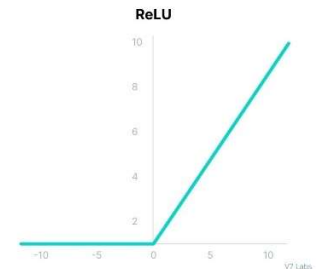
Tot i que tant el sigmoide com el tanh s'enfronten a un problema de gradient nul, **el tanh està centrat en zero** i els gradients no estan restringits a moure's en una determinada direcció. Per tant, a **la pràctica, la no linealitat del tanh sempre es prefereix a la no linealitat del sigmoide.**

Funció ReLU (Rectified Linear Unit)

Tot i que dona la impressió d'una funció lineal, ReLU té una funció derivada i permet la retropropagació (*backpropagation*) alhora que la fa computacionalment eficient.

El principal inconvenient aquí és que la funció ReLU no activa totes les neurones alhora.

Les neurones només es desactivaran si la sortida de la transformació lineal és inferior a 0.



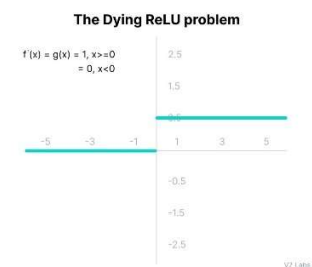
$$f(x) = \max(0, x)$$

Els avantatges d'utilitzar ReLU com a funció d'activació són els següents:

- Com que només s'activen un cert nombre de neurones, la funció ReLU és molt més eficient computacionalment en comparació amb les funcions sigmoide i tanh.
- ReLU accelera la convergència del descens de gradient cap al mínim global de la funció de pèrdua.

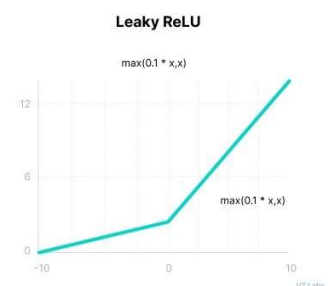
Les limitacions a què s'enfronta aquesta funció és principalment el problema Dying ReLU. El costat negatiu del gràfic fa que el valor del gradient sigui zero. Per aquest motiu, durant el procés de retropropagació, els pesos i els biaixos d'algunes neurones no s'actualitzen. Això pot crear neurones mortes que mai s'activen.

Tots els valors d'entrada negatius esdevenen zero immediatament, cosa que disminueix la capacitat del model per ajustar o entrenar correctament a partir de les dades.



Funció Leaky ReLU

Leaky ReLU és una versió millorada de la funció ReLU per resoldre el problema Dying ReLU, ja que té un petit pendent positiu a la zona negativa.



$$f(x) = \max(0.1x, x)$$

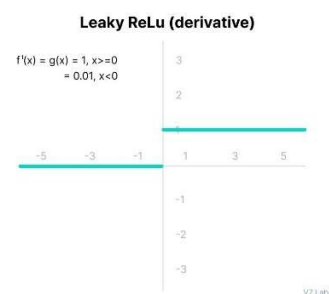
Els avantatges de Leaky ReLU són els mateixos que els de ReLU, a més del fet que permet la retropropagació (*backpropagation*), fins i tot per a valors d'entrada negatius.

Fent aquesta petita modificació per a valors d'entrada negatius, el gradient del costat esquerre del gràfic resulta ser un valor diferent de zero. Per tant, ja no trobaríem neurones mortes en aquesta regió.

La derivada de la funció Leaky ReLU.

Les limitacions a què s'enfronta aquesta funció inclouen:

- És possible que les prediccions no siguin consistents per a valors d'entrada negatius.
- El gradient per a valors negatius és un valor petit que fa que l'aprenentatge dels paràmetres del model requereixi molt de temps.



Funció Softmax

En la teoria de la probabilitat, la sortida de la funció softmax es pot utilitzar per representar una distribució categòrica, és a dir, una distribució de probabilitat sobre K resultats possibles diferents. Per tant, la funció softmax s'utilitza per a la classificació multiclasse i s'utilitza a la última capa.

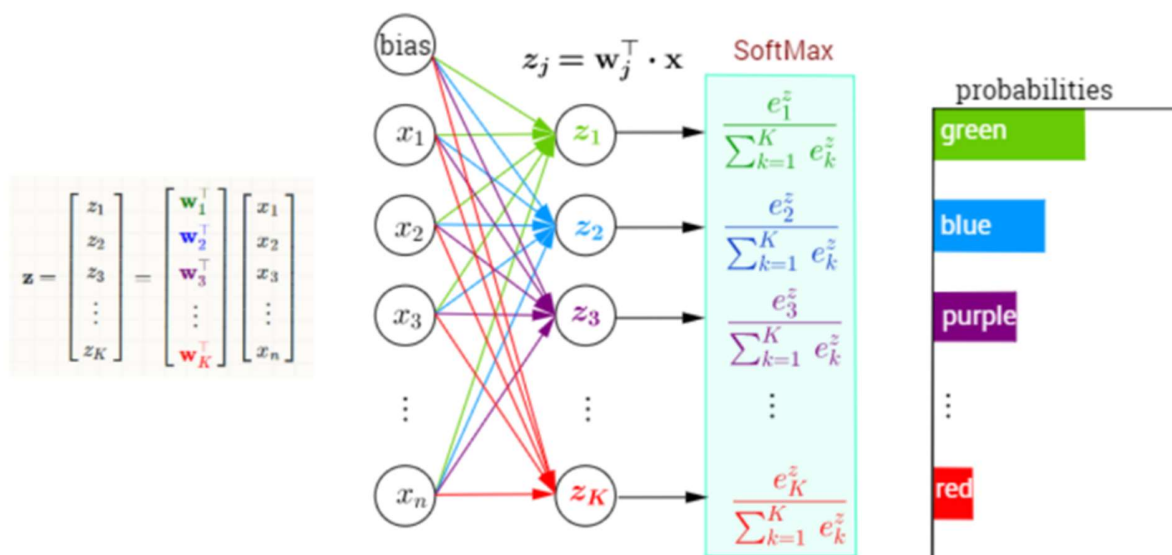
Podem considerar la funció softmax com una generalització de la funció sigmoide que permet classificar més de dues classes. La funció d'activació softmax ens garanteix que totes les probabilitats estimades es troben entre 0 i 1 i que la seva suma és igual a 1.

A nivell matemàtic, softmax utilitza el valor exponencial de les evidències calculades i, posteriorment, les normalitza de manera que sumin 1, formant així una distribució de probabilitat.

$$\sigma : \mathbb{R}^K \rightarrow [0, 1]^K$$
$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

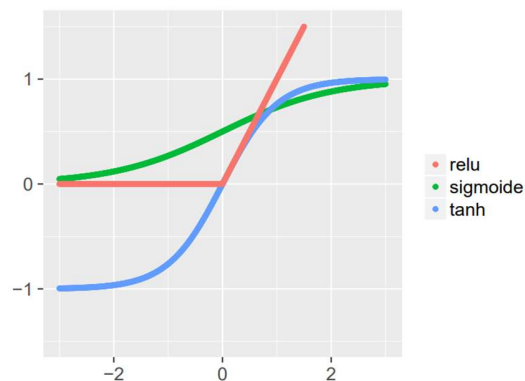
Exemple de la funció d'activació softmax:

Multi-Class Classification with NN and SoftMax Function



Quadre resum de les funcions d'activació





Name	Visualization	$f(x) =$	Notes
Linear (= Identity)		x	Not useful for hidden layers
Heaviside Step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	Not differentiable
Rectified Linear (ReLU)		$\begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	Surprisingly useful in practice
Tanh		$\frac{2}{1+e^{-2x}} - 1$	A soft step function; ranges from -1 to 1
Logistic ('sigmoid')		$\frac{1}{1+e^{-x}}$	Another soft step function; ranges from 0 to 1

Resum

- Les funcions d'activació s'utilitzen per introduir no linealitat a la xarxa.
- Una xarxa neuronal gairebé sempre tindrà la mateixa funció d'activació en totes les capes ocultes. Aquesta funció d'activació hauria de ser diferenciable per tal que els paràmetres de la xarxa s'apreguin en retropropagació.
- **ReLU és la funció d'activació més utilitzada per a capes ocultes.**
- Es busquen funcions que les seves derivades siguin simples per minimitzar el cost computacional.
- A l'hora de seleccionar una funció d'activació, cal tenir en compte els problemes que podria afrontar: gradients que desapareixen i exploten.
- Pel que fa a la capa de sortida, sempre hem de considerar el rang de valors esperats de les prediccions. Si pot ser qualsevol valor numèric (com en el cas del problema de regressió) podeu utilitzar la funció d'activació lineal o ReLU.
- Utilitzeu la funció **Softmax** o Sigmoid per als problemes de **classificació**.

- El fet de posar funcions no lineals dona expressivitat de la xarxa. Per aquest motiu una xarxa neuronal pot fer overfitting molt fàcilment si afegim moltes capes i neurones ja que tindrà tants paràmetres que trobarà una combinació de paràmetres assignarà 1 a 1 l'entrada i la sortida.

Capa	Funció activació
Cap d'entrada	Cap
Capa oculta	ReLU / Tanh/etc..
Capa sortida (binària)	Sigmoide
Capa sortida (multiclasse)	Softmax
Capa sortida (regressió)	Identitat

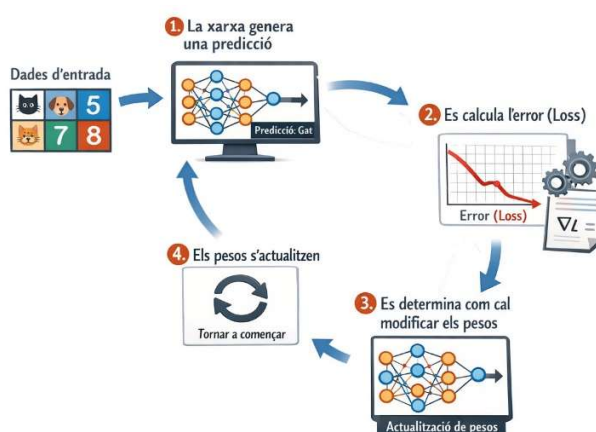
Entrenament

El procés d'entrenament el que farà serà anar **modificant els paràmetres interns** (pesos i bias) de **cadascuna de les neurones** perquè el model produeixi prediccions cada vegada més bones.

Una xarxa neuronal **no sap res a priori abans de començar** l'entrenament. Els **pesos** de les neurones **s'inicialitzen amb valors aleatoris** i, a partir d'aquí, el model millora progressivament observant els resultats i anar **minimitzant la funció de Loss**.

Aquest procés d'aprenentatge es basa en un bucle que es repeteix moltes vegades:

1. Les dades entren a la xarxa
2. La xarxa genera una predicció
3. Es calcula l'error (loss)
4. Es determina com cal modificar els pesos
5. Els pesos s'actualitzen
6. El procés es repeteix



Aquest cicle és el cor de l'aprenentatge de les xarxes neuronals on hi trobem 4 peces fonamentals:

- Forward propagation
- Funció de pèrdua/cost (Loss)
- Backpropagation
- Gradient descent (Descens de gradient)

Forward propagation

El **forward propagation** és el pas "mecànic": **agafes una entrada i la fas passar per tota la xarxa capa a capa fins obtenir una predicció**. Aquí no hi ha aprenentatge: només càlcul. **En cada capa**, el model **aplica la transformació lineal + activació** i produeix la següent representació.

Podríem veure'l com una cadena de muntatge a on cada capa rep un producte semielaborat i el transforma per passar-lo a la següent capa. Al final, l'última capa produeix la predicció (una classe, una probabilitat o un número).

Després del forward, calculem la pèrdua/loss, que és el "veredict" del model per aquella predicció.

Funció de cost (loss function)

La **funció de cost** és la mètrica que tenim per dir-li al model “com de malament/bé ho està fent”. Sense una loss, el model podria produir números, però no tindria manera de saber si millora o empitjora. En l’entrenament, l’objectiu és trobar pesos i bias que **minimitzin** aquesta funció.

Regressió

En una regressió, la Loss típica és el **Mean Squared Error (MSE)**, que penalitza més fort els errors grans perquè eleva al quadrat:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Si recordes, en les mètriques de regressió, l’MSE “castiga molt” els *outliers*. Si tenim un dataset de preus i hi ha un pis de luxe que el model prediu molt malament, el quadrat farà que aquest error tingui un impacte desproporcionat.

Una alternativa és el **Mean Absolute Error (MAE)**, que és més robust:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Classificació

En classificació, la loss més habitual és la **cross-entropy** (log loss). El model ha de donar probabilitat alta a la classe correcta. Si el model dona 0.99 a la classe correcta, “bé”; si dona 0.01, “fatal”. La cross-entropy penalitza molt els casos en què el model està segur però equivocat. Això és ideal perquè “educa” el model a calibrar probabilitats.

En binària, es formula com:

$$L = -(y \log(p) + (1 - y) \log(1 - p))$$

On p és la probabilitat predita de classe 1. En multiclasse, s’estén sumant sobre classes.

La regla professional és: tria la loss que encaixa amb la interpretació de sortida.

Sortida probabilística → CROSS-ENTROPY

Sortida numèrica → MSE/MAE.

Backpropagation (retropropagació de l'error)

El backpropagation és la clau de l'aprenentatge. La intuïció que funciona millor és: “Un cop tens l'examen corregit (la loss), has de repartir la responsabilitat de l'error entre totes les decisions que has pres”. En una xarxa, aquestes “decisions” són els pesos i els biaixos. La pregunta és: **quins pesos han contribuït més a l'error?**

El procés comença a la capa de sortida a on coneixem l'error/loss, i propaguem cap enrere fins arribar a la capa d'entrada i anar ajustant els pesos de cada neurona per la qual anem passant.

Matemàticament, això es fa amb derivades i mitjançant el **descens de gradient**.

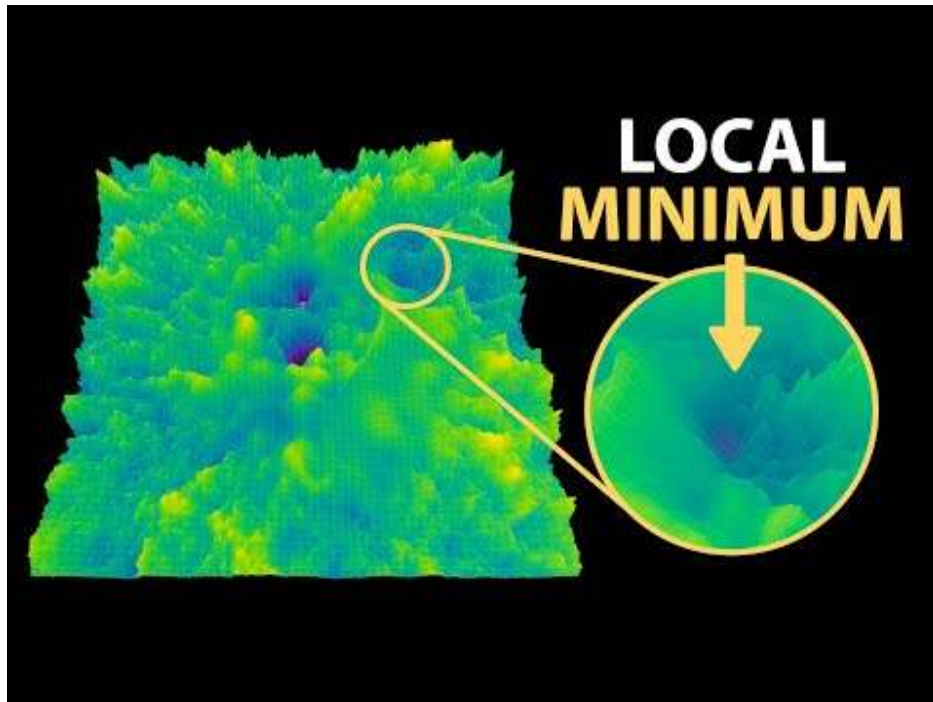
Gradient descent (Descens de gradient)

Un cop tenim definida la funció de cost el nostre objectiu és trobar els paràmetres del nostre model que minimitzin aquesta funció.

El **gradient indica** cap a la **direcció** on la **loss creix més ràpid**. Per tant, si volem reduir la loss, hem d'anar en direcció contrària al gradient.



Amb això té sentit parlar de la **derivada**, ja que la derivada d'una funció en permet saber el comportament d'una funció. Per exemple ens permet saber en quins intervals una funció creix o decreix i amb quin ritme/rapidesa ho fan.



<https://www.youtube.com/watch?v=NrO20Jb-hy0>

Optimització i actualització de pesos

L'**optimitzador** és un component clau d'una xarxa neuronal responsable d'actualitzar els paràmetres del model durant el procés d'entrenament. Concretament, la seva funció és **minimitzar la funció de pèrdua** de la xarxa neuronal ajustant els **pesos** i els **biaixos** del model.

Durant l'entrenament, la xarxa neuronal rep dades d'entrada i produeix una sortida basada en els seus paràmetres actuals. Aquesta sortida es compara amb la sortida desitjada (és a dir, la veritat de referència o ground truth) mitjançant una funció de pèrdua, que mesura com de bé està funcionant el model. L'**optimitzador** calcula aleshores els gradients de la pèrdua respecte als paràmetres del model i utilitza aquests gradients per actualitzar-los en la direcció que redueix la pèrdua.

L'optimitzador és qui decideix com han de canviar els pesos en cada pas de l'entrenament

L'elecció de l'optimitzador pot tenir un impacte significatiu en el rendiment de la xarxa neuronal, ja que cada optimitzador té avantatges i inconvenients diferents. Alguns optimitzadors populars són:

- **Descens del gradient estocàstic (SGD):** és un optimitzador senzill que actualitza els paràmetres en la direcció del gradient negatiu de la funció de pèrdua.
- **Adam (Adaptive Moment Estimation):** és un optimitzador més sofisticat que combina taxes d'aprenentatge adaptatives i *momentum* (*vola baixant per una vall, si sempre baixa en la mateixa direcció accelera*) per convergir de manera més ràpida i robusta. Ajusta automàticament la velocitat d'aprenentatge per cada pes.
- **Adagrad:** aquest optimitzador adapta la taxa d'aprenentatge de cada paràmetre basant-se en la informació històrica dels gradients.
- **RMSprop:** també adapta la taxa d'aprenentatge de cada paràmetre, però utilitza una mitjana mòbil del gradient al quadrat per fer-ho.
- **Adadelta:** és similar a RMSprop, però utilitza un mètode més sofisticat per adaptar la taxa d'aprenentatge que no requereix definir explícitament aquest paràmetre.

En conjunt, l'optimitzador és un element fonamental en el procés d'entrenament d'una xarxa neuronal, ja que ajuda el model a aprendre el conjunt òptim de paràmetres per a la tasca que s'ha de resoldre.

Regularització

En l'**aprenentatge automàtic**, la **regularització** és un conjunt de tècniques utilitzades per prevenir el **sobreajustament** (*overfitting*) en els models, incloses les xarxes neuronals. L'*overfitting* es produeix quan un model esdevé massa complex i comença a ajustar-se al soroll de les dades d'entrenament en lloc de capturar els patrons. Això pot provocar un rendiment deficient sobre dades noves que el model no ha vist abans. Les tècniques de regularització ajuden a evitar el sobreajustament limitant la capacitat del model o afegint restriccions addicionals al procés d'optimització.

En les xarxes neuronals, la regularització es pot aplicar de diverses maneres. Algunes de les tècniques més habituals són:

- **Regularització L1 (Lasso), L2 (Ridge) i Elastic Net:** Aquestes tècniques afegeixen un terme de penalització a la funció de pèrdua que incentiva que els pesos del model siguin petits.

A Keras aquest tipus de regularització es configura mitjançant el paràmetre **kernel_regularizer** de la capa que s'està afegint.

```
tf.keras.layers.Dense(  
    16,  
    activation="relu",  
    kernel_regularizer= regularizers.l1(0.005)  
)
```

Amb Keras tenim L1, L2 i Elastic NET (L1+L2)

- L1: `regularizers.l1(0.005)` → `1e-6` , `1e-4` , `1e-3`
 - L2: `regularizer.l2(0.01)` → `1e-5` , `1e-3`, `1e-2`
 - ElasticNet (L1+L2): `regularizers.l1_l2(l1=0.001, l2=0.01)`
- **Dropout:** Aquesta tècnica elimina aleatòriament algunes neurones de la xarxa durant l'entrenament, obligant les neurones restants a aprendre representacions més robustes i diverses de les dades.

```
model_dropout = tf.keras.Sequential([  
    tf.keras.layers.Dense(16, activation="relu", input_shape=(4,)),  
    tf.keras.layers.Dropout(0.5) ,  
    tf.keras.layers.Dense(8, activation="relu"),  
)
```

0.5 significa que durant l'entrenament, el 50 % de les neurones d'aquesta capa s'apaguen aleatòriament a cada batch.

- **Aturada anticipada (*Early stopping*):** Aquesta tècnica atura el procés d'entrenament abans que el model s'ajusti completament a les dades d'entrenament, evitant el sobreajustament mitjançant la recerca d'un equilibri òptim entre infrajustament (*underfitting*) i sobreajustament (*overfitting*). Aturem quan la validació empitjora.

En aquesta tècnica cal tenir en compte el paràmetre *patience*. Aquest paràmetre indica quantes epochs consecutives estem disposats a esperar sense millora abans d'aturar l'entrenament.

La podríem definir com la paciència del model.

Per exemple si tenim `patience=5`

epoch	val_loss	millora	comptador
10	0.42	Sí	0
11	0.41	Sí	0
12	0.415	No	1
13	0.417	No	2
14	0.416	No	3
15	0.418	No	4
16	0.419	No	5 → STOP!!!

Amb Keras cal tenir molt present d'activar el paràmetre `resotre_best_weights=True` perquè quan s'aturi l'entrenament es recuperin els pesos del model corresponents a l'epoch on la mètrica va ser millor i no quedar-se amb els pesos de l'últim epoch.

- **Augmentació de dades (*Data augmentation*):** Aquesta tècnica incrementa la mida del conjunt d'entrenament aplicant transformacions aleatòries a les dades d'entrada, com ara rotacions, retalls o inversions d'imatges. Això ajuda el model a ser més robust davant variacions en les dades d'entrada.

La **regularització** és una **part essencial del procés d'entrenament** de les xarxes neuronals, ja que **ajuda a prevenir el sobreajustament** i a **millorar la capacitat de generalització** del model. L'elecció de la tècnica de regularització depèn del problema específic que es vol resoldre i de les característiques del conjunt de dades.

Preprocessat

En dades tabulars, és habitual:

- **Escalat/estandardització** de variables numèriques (StandardScaler o MinMaxScaler)
- **One-hot encoding** de categòriques (si és tabular tradicional)

L'escalat és clau perquè si una feature va de 0 a 1 i una altra va de 0 a 100000, la segona dominarà els gradients i l'entrenament serà inestable. És com fer una votació on una persona té 100.000 vots i l'altra en té 1: no és "just" per al model.

Disseny de l'arquitectura

Hiperparàmetres

Les xarxes tenen molts hiperparàmetres: nombre de capes, neurones, activació, learning rate, batch size, regularització, optimitzador... i tots interactuen.

Nombre de capes (*depth* de la xarxa)

El nombre de capes de la xarxa neuronal. Normalment tindrem una capa d'entrada (input) unes quantes capes ocultes (hidden layers) i una capa de sortida (output).

Input → hidden → hidden → output

Normalment les capes seran *full connected* o denses. Normalment les capes ocultes utilitzaran la mateixa funció d'activació.

Si sobredimensionem el model amb moltes capes i neurones el més fàcil és que el nostre model sobreajusti i fàcilment caiem en overfitting.

Depenent del model però normalment hem de començar amb 2/3 capes i anar afegint a mesura del que creiem.

Les podem anar afegint mitjançant el mètode `add` o en el constructor de la xarxa mitjançant un array.

Opció 1 – Array en el constructor

```
model = keras.Sequential([
    layers.Input(shape=(X_train_sc.shape[1],)), # 4 features
    layers.Dense(16, activation="relu"), # Full connected de 16 neurones
    layers.Dense(16, activation="relu"), # Full connected de 16 neurones
    layers.Dense(3, activation="softmax") # Full connected de 3 neuornes
])
```

Opció 2 – Mètode add

```
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(X_train_sc.shape[1],))) # 4 característiques
#Afegim 2 capes de 16 neurones
model.add(layers.Dense(16, activation="relu"))
model.add(layers.Dense(16, activation="relu"))
#Afegim última capa de 3 neurones
model.add(layers.Dense(3, activation="softmax"))
```

Optimitzador

És l'algorisme que decideix com s'actualitzen els pesos a partir del gradient. En el següent enllaç trobem totes els disponibles amb [Keras](#)

Els més comuns

- SGD: simple, lent i molt didàctic

```
optimizer = tf.keras.optimizers.SGD()
```

- **Adam**: el més utilitzat. Ràpid, adaptatiu i molt robust.

```
optimizer = tf.keras.optimizers.Adam()
```

Taxa d'aprenentatge (*learning rate*)

El learning rate indica quant es modifiquen els pesos del model en cada actualització durant l'entrenament.

En Keras apareix normalment dins de l'optimitzador encara que molts d'ells en tene un per defecte:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

Epoch

Les *Epochs* són el nombre de passades completes de totes les dades d'entrenament per la xarxa neuronal.

Valors d'epochs **grans** caurem a **overfitting** i petits **underfitting**.

A priori no sabem el nombre òptim d'epochs. Per tant el que fem és observar és la corba de Loss i de Validació.

Aquest hiperparàmetre es configura en el mètode fit del model.

```
model.fit(  
    epochs = 20,  
    batch_size=16  
)
```

Batch Size

El *batch size* és el nombre de mostres que el model utilitza abans de fer una actualització dels pesos. És a dir no s'actualitzen els pesos després de cada mostra, sinó que s'espera *batch size* per actualitzar els pesos.

Per exemple si tenim 100 mostres amb un batch size de 10

Cada epoch es divideix en 10 batches de 10 mostres.

Normalment s'utilitzen potències de dos (16,32,64,128,256,...) com a valors del hiperparàmetre batch

size

Aquest hiperparàmetre es configura en el mètode fit del model.

```
model.fit(  
    epochs = 20,  
    batch_size = 16  
)
```

Validació del model i seguiment de mètriques

Un cop definida l'arquitectura d'una xarxa neuronal i configurat l'entrenament, el següent pas fonamental és validar el comportament del model i fer-ne un seguiment al llarg del temps. Aquest procés ens permet entendre si el model està aprenent correctament, si generalitza bé o si, pel contrari, està començant a memoritzar les dades.

En xarxes neuronals no n'hi ha prou amb entrenar el model i mirar el resultat final. El valor real està en observar com evoluciona l'aprenentatge epoch rere epoch.

Per fer aquest seguiment amb Keras tenim els següents valors:

- loss
- accuracy
- val_loss
- val_accuracy

loss i val_loss

El valor **loss** indica **quant s'equivoca el model** sobre les dades d'entrenament. És el resultat d'aplicar una funció de pèrdua (com ara `mse`, `binary_crossentropy` o `categorical_crossentropy`) a les prediccions del model.

Per altra banda, **val_loss** mesura exactament el mateix error, però **sobre el conjunt de validació**, és a dir, dades que **no s'han utilitzat per ajustar els pesos**.

Entrem en overfitting quan la loss baixa, però la val_loss s'estanca o comença a pujar.

A més de l'error, habitualment també mesurem la **precisió (accuracy)**, que indica el **percentatge de prediccions correctes**.

`accuracy` → encerts sobre dades d'entrenament

`val_accuracy` → encerts sobre dades de validació

Guardar i restaurar un model

Un cop hem entrenat un model i l'hem analitzat que els resultats ens satisfan podem guardar-lo amb Keras mitjançant el mètode `save`

```
model.save("<nom_model>", save_format="tf")
```

El paràmetre `save_model="tf"` és per indicar-li a Keras que guardi el model utilitzant el model de TensorFlow.

Per restaurar el model utilitzarem el mètode `load_model`

```
model = tf.keras.models.load_model("<nom_model>")  
Y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Utilització de callbacks

El mètode `fit()` accepta un argument anomenat `callbacks` que permet especificar una llista d'objectes a que Keras cridarà abans i després de processar un batch

Això serveix per guardar punts de control durant un entrenament d'un model molt gran que trigui hores o dies. D'aquesta manera podem guardar aquests punts de control amb regularitat per si falla l'ordinador.

Frameworks de Xarxes Neuronals

De la mateixa manera que scikit-learn és una de les llibreries a on trobem infinitat de models de Machine Learning. Inclús un perceptró, existeixen altres llibreries i frameworks més especialitzats. Els més populars actualment són TensorFlow, PyTorch i Keras.

TensorFlow

TensorFlow és la llibreria de *Deep Learning* més popular i amb més contribucions. Va ser **desenvolupada** originalment per **Google Brain** per a l'ús intern en productes d'IA de Google i es va fer **open source** el 2015 i des de llavors s'ha convertit en l'estàndard de facto.

TensorFlow és la base de productes de Google com Google Translate i les API de machine learning de Google Cloud Platform.



TensorFlow es va dissenyar perquè es pugues paral·lelitzar, i per això té un rendiment excel·lent en entorns distribuïts.

TensorFlow proporciona **API** per a **Python, C++, Java i altres llenguatges**

En Python es pot instal·lar fàcilment mitjançant pip

```
pip install tensorflow
```

PyTorch

PyTorch és una llibreria *Deep Learning* creada pel laboratori de recerca en intel·ligència artificial de Facebook (META). A diferència de TensorFlow, PyTorch està dissenyat per imitar Python natiu, fet que el fa molt intuïtiu per a programadors de Python.



Es pot instal·lar amb:

```
conda install pytorch torchvision -c pytorch
```

Keras

Keras és la llibreria de Deep Learning de més alt nivell. Actualment està integrada dins TensorFlow i es considera el wrapper per



exel·lència de TensorFlow a partir de la versió 2.4, però en principi pot

funcionar sobre altres llibreries com MXNet o Theano. Es va publicar com un projecte de codi obert el 2015, però va ser desenvolupat per François Chollet com a part d'un procés d'investigació.

Quan instal·lem TensorFlow també s'instal·la Keras de forma automàtica.

Es pot instal·lar amb:

```
pip install keras
```

Experimentació amb TensorFlow

[TensorFlow Playground](#) és una aplicació web de visualització interactiva, escrita en JavaScript, que ens permet simular xarxes neuronals densament connectades que s'executen al nostre navegador i veure els resultats en temps real.

Permet afegir fins a 6 capes internes amb fins a 8 neurones per capa. En entrenar la xarxa neuronal, podem veure si ho estem aconseguint o no mitjançant la mètrica "Training loss", és a dir, la funció de pèrdua per a les dades d'entrenament. Posteriorment, per comprovar que el model generalitza correctament, també s'ha d'aconseguir minimitzar la "Test loss", és a dir, l'error calculat per la funció de pèrdua per a les dades de test.

Es pot jugar amb aquest simulador realitzant els següents canvis:

- Augmenta la proporció de dades d'entrenament respecte a les de test fins al 70%
- Deixa una única capa interna amb una sola neurona
- Afegeix 2 neurones més a la capa interna (de manera que tingui 3 neurones)

- Canviar el conjunt de dades pel que té 4 zones quadrades diferents
- Canviar el conjunt de dades pel que té forma de remolí (necessitarà més capes amb més neurones)
- Canviar els valors dels hiperparàmetre (batch size, learning rate, funcions d'activació regularització,...)
- Afegir soroll a les dades d'entrada

Conclusions:

- Poques neurones a les capes ocultes provocaran infrajustament (*underfitting*).
- Massa neurones a les capes ocultes provocaran sobreajustament (*overfitting*) —la xarxa neuronal té més capacitat de processament d'informació que la quantitat d'informació continguda en el conjunt d'entrenament, que no és suficient per entrenar totes les neurones de les capes ocultes— i també un temps de processament molt més elevat.

Implementació d'una xarxa neuronal amb Keras

Keras és l'API de Deep Learning a al nivell de TensorFlow. Permet crear, entrenar, avaluar i executar tot tipus de xarxes neuronals.

Exemple de Xarxa Neuronal - Regressió

Ajust d'hiperparàmetres

La flexibilitat de les xarxes neuronals és també un dels principals inconvenients. Hi ha molts hiperparàmetres per ajustar. El número de capes, el número de neurones, els tipus de funció d'activació per cada capa, la lògica d'inicialització de pesos, el learning rate, etc.... Com podem saber quina és la millor combinació d'hiperparàmetres per el nostre problema?

Una opció és convertir el model Keras a un estimador de Scikit-Learn i utilitzar GridSearchCV o RandomizedSearchCV per perfeccionar els hiperparàmetres tal i com hem fet amb altres models supervisats. Per això podem utilitzar les classes **KerasRegressor** i **KerasClassifier** de la biblioteca SciKeras (<https://github.com/adriangb/scikeras>)

Però la millor opció és utilitzar la biblioteca **Keras Tuner**, biblioteca per models de Keras

Instal·lació

```
pip install -q -U keras-tuner
```

Construeix un mètode capaç de construir un model

```
def build_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    n_neurons = hp.Int("n_neurons", min_value=16, max_value=256)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,
sampling="log" )
    optimizer = hp.Choice("optimizer", values=["adam", "sgd"])
    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    else:
        optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)

    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Flatten())
    for _ in range(n_hidden):
        model.add(tf.keras.layers.Dense(n_neurons, activation="relu"))
    model.add(tf.keras.layers.Dense(10, activation="softmax"))
    model.compile(loss="sparse_categorical_crossentropy",
optimizer=optimizer, metrics=["accuracy"])

    return model
```

Crida el procediment de creació del model mitjançant el mètode RandomSearch de keras-tuner

```
random_search_tuner = kt.RandomSearch(  
    build_model, objective="val_accuracy", max_trials=5, overwrite=True,  
    directory="my_model_results", project_name="my_rnd_search", seed=45)  
  
random_search_tuner.search(X_train, y_train, epochs=10,  
                           validation_data=(X_valid, y_valid))
```

En el codi anterior executem 5 proves. Per cada prova, es construeix un model utilitzant hiperparàmetre de manera aleatòria dins els rangs definits. Entren el model durant 10 epochs i guarda en un subdirectori anomenat my_model_results/my_rndsearch

Com que hem configurat `overwrite=True` es sobreesciu l'entrenament. Si volem que s'executi el codi una segona vegada i que l'ajustador continui a on ho va deixar hem de configurar `overwrite=False` i `max_trials = 10`. Això executarà 5 proves més.

Obtenim els millors models

```
top3_models = random_search_tuner.get_best_models(num_models=3)  
best_model = top3_models[0]
```

També podem cridar al mètode `get_best_hyperparameters(num_trials=3)` per obtenir els `kt.HyperParameters` dels millors models.

```
>>> top3_params = random_search_tuner.get_best_hyperparameters(num_trials=3)  
>>> top3_params[0].values # Retorna els millors valors dels hiperparàmetres.
```

Número de capes ocultes (*Hidden layers*)

Per a molts problemes, pots començar simplement amb **una sola capa oculta** i obtenir resultats raonables. Durant molt de temps s'havia pensat que no calia explorar xarxes neuronals més profundes sinó

augmentar el número de neurones.

Però s'ha vist que les xarxes profundes (més capes) tnen una eficiència de paràmetres molt superior. Poden modelar funcions complexes utilitzant moltíssimes menys neurones.

Les dades del món real sovint tenen una estructura jeràrquica, i les xarxes neuronals profundes aprofiten aquest fet:

- Les **capas ocultes inferiors** modelen estructures de baix nivell (per exemple, segments de línia amb diferents formes i orientacions).
- Les **capas intermitges** combinen aquestes estructures de baix nivell per modelar estructures de nivell mitjà (per exemple, quadrats o cercles).
- Les **capas superiors** i la **capa de sortida** combinen aquestes estructures intermèdies per modelar estructures d'alt nivell (per exemple, cares).

Aquesta arquitectura jeràrquica no només ajuda a convergir cap a una bona solució, sinó que també millora la seva capacitat de generalització.

Per exemple, si ja hem entrenat un model per reconèixer cares en imatges i ara volem entrenar una nova xarxa neuronal per reconèixer pentinats, podem iniciar l'entrenament reutilitzant les capes inferiors de la primera xarxa. En lloc d'inicialitzar aleatòriament els pesos i biaixos de les primeres capes, podem assignar-los els valors apresos pel model anterior. D'aquesta manera, la nova xarxa no haurà d'aprendre des de zero totes les estructures de baix nivell comunes a la majoria d'imatges; només haurà d'aprendre les estructures d'alt nivell (per exemple, els pentinats). Això s'anomena **aprenentatge per transferència** (*transfer learning*).

Número de neurones per capa

El nombre de neurones de les capes d'entrada i sortida estan determinades pel tipus d'entrada i sortida. Per exemple en el dataset MNIST necessitem $28 \times 28 = 784$ neurones d'entrada i 10 neurones de sortida.

Pel que fa a les **capas ocultes** era habitual dimensionar-les **en forma de piràmide**. Per exemple, una xarxa típica per a MNIST podia tenir **tres capas ocultes**: 300 neurones → 200 neurones → 100 neurones.

Aquesta pràctica s'ha abandonat en gran mesura, ja que sembla **que utilitzar el mateix nombre de**

neurones a totes les capes ocultes funciona igual de bé, o fins i tot millor, i només cal ajustar un únic hiperparàmetre en lloc d'un per capa. Per exemple, totes les capes ocultes podrien tenir 150 neurones. Tot i així, segons el conjunt de dades, de vegades pot ajudar que la primera capa oculta sigui més gran que les altres.

En general, obtindràs més rendiment augmentant el nombre de capes que no pas el nombre de neurones per capa.

És senzill fer un model amb més capes i neurones de les que realment necessites i després utilitzar early stopping per evitar el sobreajustament (i altres tècniques de regularització, com el dropout). Aquest mètode s'ha anomenat l'enfocament dels “pantalons elàstics”: en lloc de perdre temps buscant uns pantalons que s'ajustin perfectament a la teva talla, simplement utilitza uns pantalons grans i elàstics que s'ajustin sols a la mida adequada.

Learning Rate, Batch Size i altres hiperparàmetre

El nombre de capes ocultes i neurones no són els únics hiperparàmetres que es poden ajustar en una xarxa neuronal. Aquests són alguns dels més importants i alguns consells per configurar-los:

- **Taxa d'aprenentatge (*learning rate*):** probablement és l'hiperparàmetre més important. En general, la taxa òptima és aproximadament la meitat de la taxa màxima (és a dir, la taxa per sobre de la qual l'algorisme divergeix). Una **estratègia senzilla** és començar amb un **valor alt** que faci divergir l'entrenament, **dividir-lo per 3 i repetir fins que l'entrenament deixi de divergir**. En aquest punt, normalment estaràs a prop del valor òptim. De vegades també és útil reduir la taxa durant l'entrenament.
- **Optimitzador:** escollir un optimitzador millor. Mirar el capítol 11 del llibre d'Aurélien Géron
- **Mida del batch:** pot tenir un impacte significatiu en el rendiment i el temps d'entrenament. En general, la **mida òptima sol ser inferior a 32**. Un batch petit fa que cada iteració sigui ràpida; tot i que un batch gran proporciona una estimació més precisa del gradient, en la pràctica això no sol ser crucial. Tanmateix, **una mida superior a 10 aprofita millor les optimitzacions de maquinari i programari** (especialment en multiplicacions de matrius) i accelera l'entrenament. Si s'utilitza **Batch Normalization**, el batch no hauria de ser massa petit (normalment no inferior a 20).
- **Funció d'activació:** en general, **ReLU** és una bona opció per defecte per a les capes ocultes. Per a

la capa de sortida, dependrà de la tasca a realitzar.

- **Nombre d'iteracions d'entrenament:** normalment no cal ajustar-lo manualment; és millor utilitzar **early stopping**.

Per a més bones pràctiques, és recomanable llegir [l'article de Yoshua Bengio \(2012\)](#), que ofereix moltes recomanacions pràctiques per entrenar xarxes profundes.

Tipus de Xarxes Neuronals

FNN /MLP (Feedforward Neural Network /Multilayer Perceptron)

Una xarxa neuronal directa (FNN) és el tipus més bàsic de xarxa neuronal artificial en l'aprenentatge automàtic.

La informació flueix en una direcció, des de l'entrada fins a la sortida, sense cap bucle/cicle o retroalimentació.

Per què podem utilitzar les xarxes FNN?

- **Aproximador universal de funcions:** Les xarxes neuronals *feedforward* (FNN) poden aproximar qualsevol funció contínua, cosa que les fa molt potents tant per a tasques de **regressió** com de **classificació**.
- **Reconeixement de patrons:** Són molt eficients aprenent patrons dins les dades, com ara en imatges, text o dades estructurades.
- **Generalització:** Un cop entrenades, generalitzen bé a dades no vistes, sobretot si aquestes són similars a les dades amb què s'han entrenat.

Quan utilitzar-les?

- **Classificació:** Quan necessitem classificar dades en diferents categories (p. ex., categories de productes en una botiga minorista).
- **Regressió:** Quan necessitem predir valors continus (p. ex., predir vendes a partir del rendiment

passat).

- **Extracció de característiques:** Quan necessitem aprendre relacions entre característiques d'entrada (p. ex., relacions entre la demografia dels clients i el seu comportament de compra).

Les FNN són adequades en els següents escenaris:

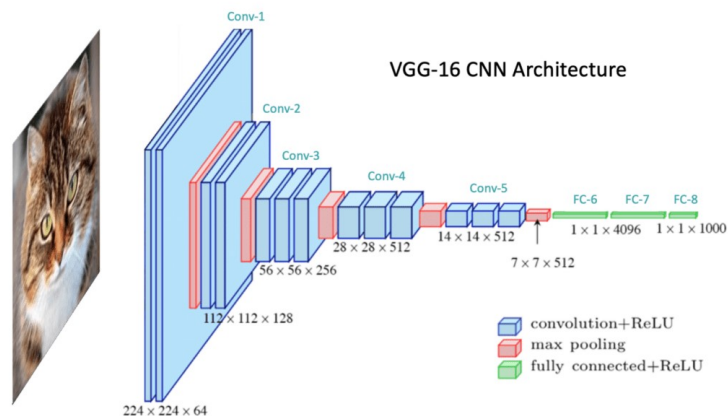
- Problemes d'aprenentatge supervisat: tasques com classificació (p. ex., identificar segments de clients) i regressió (p. ex., predir vendes futures).
- Dades estructurades: quan tenim dades tabulars, com registres de transaccions de clients, dades de vendes, etc.
- Reconeixement de patrons bàsic: quan les relacions entre entrada i sortida són relativament directes (p. ex., predir els ingressos d'una botiga a partir del trànsit de persones).

A vegades aquest tipus de xarxes

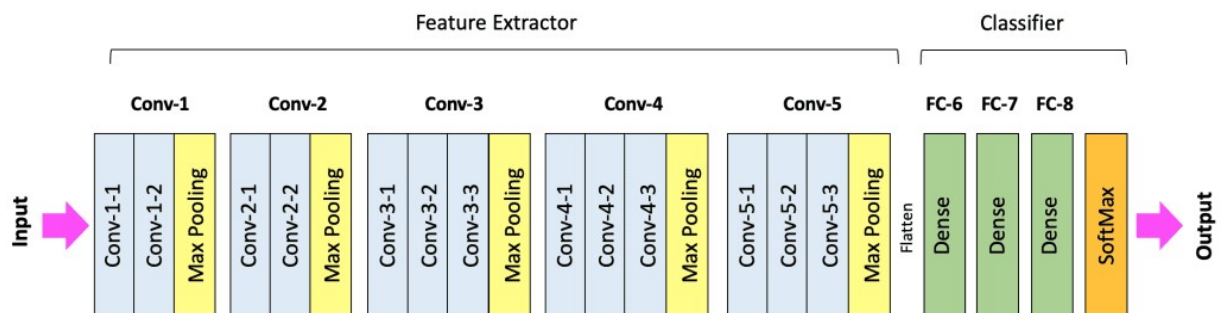
CNN (Convolutional Neural Network)

Una FNN mira tots els píxels com si fossin números sense relació. **Una imatge no és només una llista de píxels**, sinó que aquests **tenen relacions espacial entre ells**. Si movem un objecte lleugerament cap a l'esquerra, continua essent el mateix objecte. Aquest tipus de xarxes estan pensades per dades amb estructura espacial i popularment s'utilitzen per tractament d'imatges.

Podem imaginar una **CNN com un sistema de visió artificial** que **primer detecta línies**, després combina **línies en formes**, i **finalment reconeix objectes complets**. Aquesta jerarquia de representació és el que fa que les CNN siguin tan potents.



<https://learnopencv.com/>



Vídeo explicatiu de com funciona una xarxa convolucional 2D: <https://youtu.be/yb2tPt0QVPY>

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

RRNN (Recurrent Neural Netowrks)

Quan treballem amb dades seqüencials, com text o sèries temporals, necessitem un model que entengui l'ordre. Aquí apareixen les RNN. Aquest tipus de xarxa introdueix el concepte d'estat intern, que es

transmet d'un pas temporal al següent.

Aquestes xarxes neuronals **afegeixen algunes connexions d'una capa interior a l'entrada** per tenir una mica de feedback. Això es fa per **dotar a la xarxa de certa memòria**.

Podem imaginar una RNN com una persona llegint una frase paraula per paraula, recordant el context del que ha llegit abans. Aquest record és l'estat ocult. Cada nova entrada modifica aquest estat.

Aquestes xarxes s'han utilitzat en anàlisi de sentiments, traducció automàtica i predicció de sèries temporals. No obstant això, tenen un problema conegut com a “*vanishing gradient*”, que **dificulta recordar informació llunyana en la seqüència**.

```
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=(50, 1)),
    tf.keras.layers.Dense(1)
])
```

LSTM (Long Short-Term Memory)

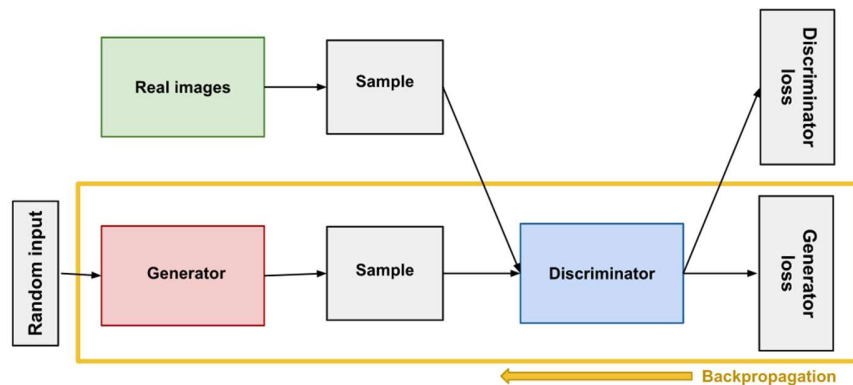
Les LSTM són una evolució dels RNN dissenyades per solucionar el problema de la memòria curta (*vanishing gradient*). És un tipus especial de RNN. Tenen la característica per solucionar un problema que tenen les xarxes neuronals que és la disminució del gradient. Incorporen mecanismes interns anomenats portes que decideixen quina informació s'ha d'oblidar, quina s'ha de guardar i quina s'ha de transmetre. En entorns professionals, les LSTM s'utilitzen en predicció meteorològica, anàlisi financer, processament de llenguatge natural i sistemes de predicció industrial basats en sensors IoT.

```
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(64, input_shape=(100, 1)),
    tf.keras.layers.Dense(1)
])
```

GANs

És un dels tipus de xarxa més innovadora i més potent en Deep Learning. És una xarxa dissenyada per generar no per classificar. Tenen 3 blocs: una és la xarxa que volem que digui les coses, una altra que processa les nostres dades reals i una tercera que combina les dues.

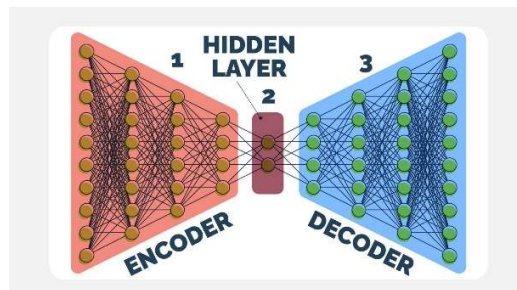
Volem generar cares. Generam una sortida (CNN) aleatori, però tenim una altra xarxa FCN que intenta quan jo li passo una imatge real la xarxa detecta si és real o generada per mi. Aquest error anem actualitzant la que genera la imatge perquè generi menys error i sigui més fidedigne. A final arriba un punt que el discriminant ja no distingeix entre la imatge real o de la generada. I al final tenim un model capaç de generar imatges.



Això és molt maco, però fer una xarxa d'aquest tipus i que funcioni no és fàcil. Ja que tenim un generador i un discriminador.

Xarxes Autoencoder

Els autoencoders són xarxes dissenyades per aprendre una representació comprimida de les dades. Els autoencoders consten d'una xarxa de codificadors que comprimeix les dades d'entrada i una xarxa de descodificadors que intenta reconstruir les dades originals a partir de la representació comprimida. Funcionen com un sistema de compressió intel·ligent. L'entrada es redueix a una capa central anomenada "bottleneck" i després es reconstrueix.



Dissenyat per aprendre una representació comprimida de les dades d'entrada, que després es pot utilitzar per a tasques com ara la reducció de soroll d'imatges, la compressió de dades i la detecció d'anomalies.

```
input_layer = tf.keras.layers.Input(shape=(20,))
encoded = tf.keras.layers.Dense(10, activation='relu')(input_layer)
decoded = tf.keras.layers.Dense(20, activation='sigmoid')(encoded)

autoencoder = tf.keras.Model(input_layer, decoded)
autoencoder.compile(optimizer='adam', loss='mse')
```

Transformers

Els Transformers van ser introduïts el 2017 amb el famós article “Attention is All You Need”. El seu impacte ha estat tan gran que han substituït pràcticament les LSTM en processament de llenguatge natural. La idea revolucionària és el mecanisme d’atenció (self-attention). En lloc de processar les dades seqüencialment com fan les RNN, el Transformer analitza tota la seqüència alhora i calcula quines parts són més rellevants entre si.

Imaginem que llegim aquesta frase: “El professor va corregir l’examen perquè estava mal redactat.” Quan arribem a “estava”, el model ha d’entendre a què fa referència. Les LSTM ho fan seqüencialment. El Transformer mira tota la frase simultàniament i calcula relacions de pes entre paraules.

CNN Convolutional Neural Networks

Les xarxes neuronals FNN *full-connected*. Funcionen molt bé amb dades tabulars, però tenen una gran limitació. No apofiten l'estructura interna de les dades i per aquestes una observació del conjunt de dades té la mateixa relació amb una altra observació del mateix conjunt de dades.

Imaginem que tenim una imatge de 28x28 píxels. Una FNN veu les dades com un vector de 784 números.

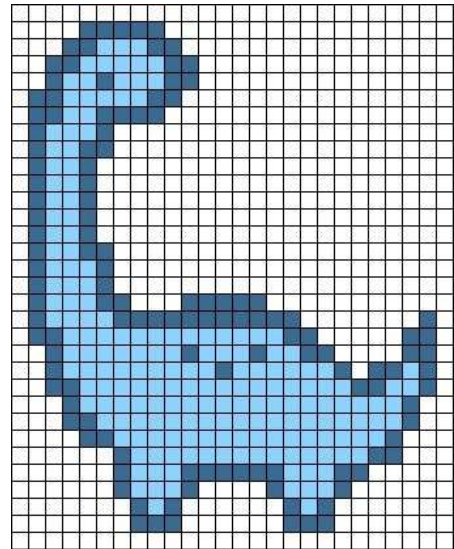
Per ella:

- El píxel (0,0) no té cap relació especial amb el (0,1)
- No sap que els píxels veïns formen formes
- No entén que hi ha una estructura espacial

És com si li donéssim un puzzle desmuntat i li diguéssim que identifiqui què representa sense poder mirar la imatge sencera.

Les xarxes neuronals convolucionals estan dissenyades per processar dades en forma de quadrícula. D'aquesta manera aprofita l'estructura local de les dades per aprendre representacions compactes i efectives capturant patrons espacials que permeten distingir part del conjunt d'entrada.

S'utilitzen principalment en tasques de visió per computador com: classificació d'imatges, detecció d'objectes, reconeixement facial o diagnòstic mèdic.



Capa convolucional (Convolutional layer)

Imaginem que tenim una lupa petita o finestra (un filtre/kernel). Aquesta finestra normalment té un mida de 3x3.

Kernel_size = (3x3)

[w11 w12 w13]

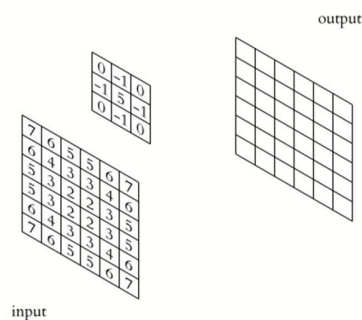
[w21 w22 w23]

[w31 w32 w33]

Cada posició de la matriu conté una sèrie de pesos inicialitzats. Si només tenim en compte només la part 2D tindrem 9 pesos, però si la imatge té RGB (3 canals), el kernel realment és de $3 \times 3 \times 3 = 27$ pesos. O sigui

una matriu de 3x3 per cada canal RGB.

El que fem amb aquest kernel és anar-lo movent per sobre de la imatge i a cada posició, la finestra fa una mena de “puntuació”. Aquesta puntuació el que fa és **un producte escalar local multiplicant cada valor de la imatge per el pes del kernel i ho sumem**. El resultat és un píxel o valor de sortida. Repetint això per totes les posicions obtenim el que s’anomena un mapa de característiques (*feature map*).



Hiperparèmtres importants: kernel, strid, padding

Kernel size (mida del filtre)

El més típic és 3×3, perquè és un bon equilibri: petit, barat, i combinant diverses convolucions 3×3 pots obtenir un camp receptiu efectiu gran (i amb més no-linealitats pel camí). En pràctica moderna, moltes arquitectures prefereixen apilar 3×3 abans que usar 7×7 a tot arreu.

Stride (pas)

El stride és quant “saltes” cada cop que mous el filtre. Stride 1 vol dir que el filtre recorre píxel a píxel. Stride 2 vol dir que “fa salts” i per tant redueix la resolució (downsampling). És una manera d’abaixar mida sense pooling (o combinat amb pooling). El que no té sentit és tenir un strid més gran que el tamany del kernel perquè hi haurà píxels que no els tractarem.

Padding

Quan fem una convolució, el kernel necessita “posar-se” sobre la imatge. Però què passa a les vores? El filtre no pot centrar-se bé si no hi ha píxels al voltant. Aquí és on apareix el *padding*.

zero

No afegim es a la imatge , per tenim els inconvenient que les vores es processen menys i la mida de la imatge es redueix. És l'opció per defecte

```
Conv2D(32, (3,3), padding="valid")
```

Padding="same"

Si volem conservar la mida (amb stride 1), uses padding="same", que afegeix zeros al voltant.

Això permet que la sortida tingui la mateix mida que l'entrada si stride=1

	0 0 0 0
1 2 3	0 1 2 3 0
4 5 6	0 4 5 6 0
7 8 9	0 7 8 9 0
	0 0 0 0

```
Conv2D(32, (3,3), padding="same")
```

Reflection padding - Reflect /Mirroring / Folding

Moltes vegades si afegim zeros estem distorsionant molt la imatge si tenim informació (valors alts) a les vores. Per no distorsionar la imatge "dobleguem" la imatge sobre el marge per mantenir la continuïtat estadística i suavitat a la frontera.

	5 4 5 6 5
1 2 3	2 1 2 3 2
4 5 6	5 4 5 6 5
7 8 9	8 7 8 9 8
	5 4 5 6 5

```
import tensorflow as tf

x_padded = tf.pad(x, [[0,0],[1,1],[1,1],[0,0]], mode="REFLECT")
```

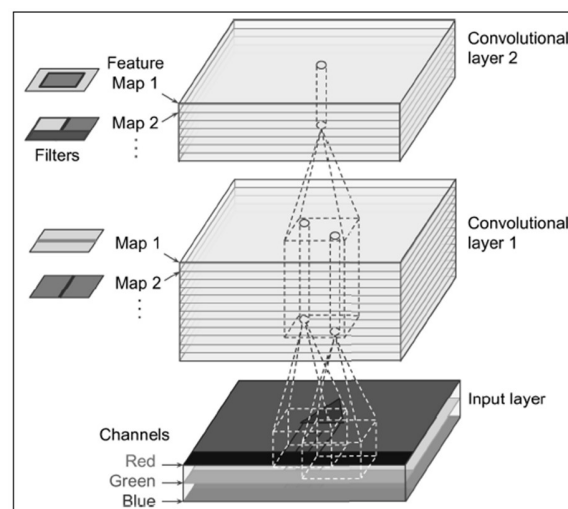
Apilar múltiples mapas de características

Cada nou píxel ara ens està expressant els 9 de la finestra. Per tant ens està expressant un valor referent al contorn d'aquells píxels. L'espai o el lloc de cada píxel importa.

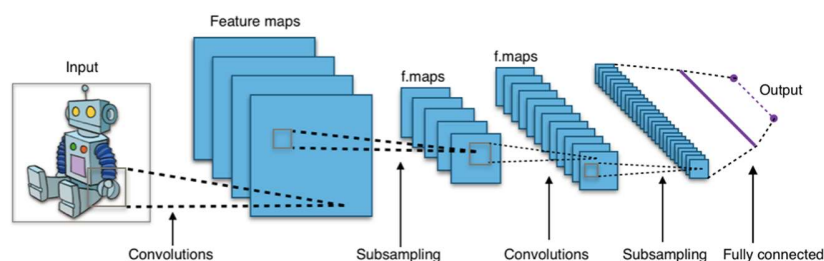
La operació de convolució vol que adaptar els *kernels* perquè detectin coses a la imatge. Detectar: formes, contorns, colors, textures, etc...

Tal com hem dit la nova imatge generada és un mapa de característiques o *feature map*. I si en comptes d'una convolució en fem 32 o 64? Tindrem 32 o 64 maps de característiques diferents.

Per tant podem tenir 32 maps de característiques diferents. Per exemple un mapa ens agafa les línies verticals, els altres les horitzontals, etc...



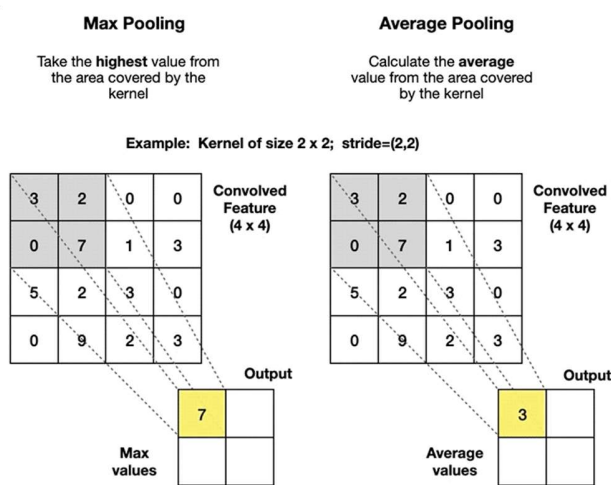
Capa de *Pooling* (*Pooling layer*)



Convolutional neural network representation | [Source](#)

Un cop entès el procés de convolució, és fàcil entendre el procés d'agrupació o *pooling*. El seu objectiu és realitzar submostres, és a dir, reduir la imatge per disminuir la càrrega computacional, l'ús de memòria i el número de paràmetres, però mantenint les característiques rellevants (vores, colors,...).

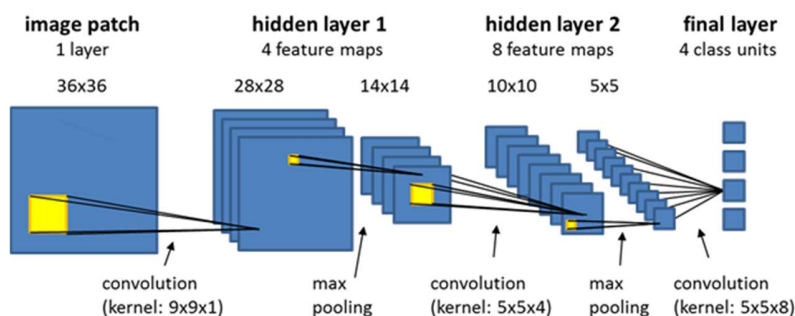
La capa d'agrupació funciona independentment en cada porció de profunditat de l'entrada. La redimensiona espacialment, utilitzant el **màxim** o la **mitjana dels valors d'una finestra** que s'ha desplaçat sobre les dades d'entrada.



Max and Avg Pooling Layers | [Source](#)

En l'exemple anterior reduïm un mapa de característiques 4x4 a 2x2.

De la mateix manera que en procés convolucional, cada neurona d'una capa de *pooling* està connectada a les sortides d'un número limitat de neurones de la capa anterior, però en el procés de *pooling* no apliquem cap nucli (pesos) a les dades d'entrada, sinó que simplement utilitzem una operació matemàtica com màx o avg. Aquesta operació d'agrupació (*pooling*) s'aplica independentment a cada canal de les dades d'entrada, o sigui a cada mapa de característiques.



Tenim diferents MaxPoolings.

- MaxPooling1D: s'utilitzen amb dades seqüencials. Dades de sèries temporals, audios, texts (embeddings),...
- MaxPooling2D: S'utilitzen en imatges
- MaxPooling3D: S'utilitzen en vídeos (frames + height + width), imatges mèdiques 3D,

Amb aquesta representació veiem que en les etapes de convolució i pooling no es redueixen el nombre de canals.

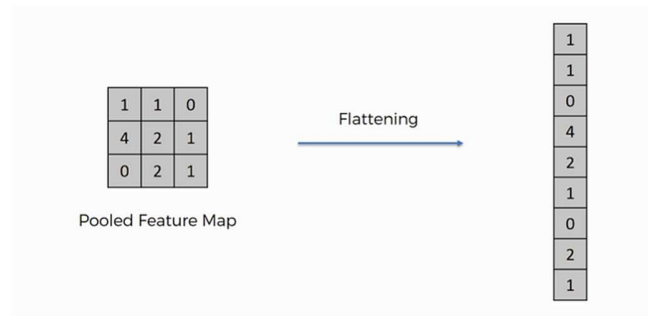
A mesura que anem avançant a la xarxa anem tenint imatges més petites, per tant podrem tenir més capes sense assumir cost computacional. Per tant a mesura que anem avançant podem tenir cada vegada més capes.

Capa d'aplanament (*Flatten Layer*)

Després que les **capes convolucionals** i **d'agrupació** hagin **extret les característiques rellevants** de la imatge d'entrada, hem de convertir aquest mapa de característiques d'alta dimensió en un format adequat per alimentar capes completament connectades per entra a la fase de decisió o classificació. Aquí és com intervé la capa d'aplanament.

Imaginem que tenim una quadrícula de dades (com ara píxels en un mapa de característiques) i volem alinear tots aquests punts de la quadrícula en una sola línia llarga.

Això és el que fa l'aplanament. Agafa **tot el mapa de característiques** i el reorganitza en **un sol vector llarg**.



Ara bé tot i que l'**aplanament** canvia la forma de les dades, **no fa cap canvi a la informació real**. Una capa densa (FNN) espera un vector 1D per cada mostra.

Simplement posem un capa al darrera l'altra respectant l'ordre intern.

Per exemple si tenim 3 mapes de 3 x 3 tindrem un vector de 27 posicions a on les 9 primeres posicions del vector correspondran a les 9 cel·les del primer mapa. Els següents 9 valors del vector seran els 9 valors del mapa 2 i per últim els valors del mapa 3.

Resum

